

Chapter 12

Post-Quantum Cryptography



Abstract This chapter is devoted to post-quantum cryptography (PQC) concepts. Following the introduction, the following lattice-based cryptography concepts are described: module learning with errors (LWE)-based cryptography, ring LWE-based cryptography, lattice-based digital signatures, and lattice-based hash functions. The code-based cryptography concepts are described next including McEliece encryption scheme, Niederreiter cryptosystem, and quasi-cyclic (QC)-LDPC coding-based encryption. In the following section, the hybrid QKD-PQC concepts are introduced, in which the QKD and PQC are operated in a cooperative fashion. In this scheme, the raw key transmission is performed over QKD subsystem, while information reconciliation over the PQC subsystem. The following topics are described in this section: hybrid PQC-QKD encryption, spatially coupled LDPC coding-based McEliece encryption subsystem, and hybrid twin field (TF)-QKD-PQC encryption. The focus is then moved to the hash-based cryptography. After introducing the basic concepts from the hash-based cryptography including Lamport's one-time signature scheme, two relevant schemes are described: Winternitz one-time signature (W-OTS) scheme and Merkle signature scheme (MMS) that is applicable to many-times signatures. After that we described relevant multivariate cryptography concepts.

12.1 Introduction

The quantum key distribution (QKD), described in previous chapters, employs fundamental physics to guarantee security [1–6] rather than the unproven mathematical assumptions for computational security. While the first satellite-to-ground QKD demonstration [5] has given momentum to QuCom research, there remain several barriers to widespread applications. To illustrate, channel loss limits both the secret-key rate (SKR)/single-mode and achievable distance in a rate-loss tradeoff. To overcome these barriers, recently following two approaches have become popular: (i) the development of quantum relays [7] and (ii) the introduction of the trusted relays concept [8]. However, the quantum relays require quantum memories of long-duration and high-fidelity entanglement, which are not commercially available. On

the other hand, in practical applications it is difficult to verify the trust for the relay between any two nodes in an optical network. QKD can indeed be used to build the future secure networks. Unfortunately, the SKRs for current discrete variable (DV) QKD systems are very low so that the corresponding quantum key “pool” storing the secret and secure keys will often be empty, thus hampering the operation of these networks. Continuous variable (CV)-QKD systems do not exhibit the dead time problem and can be used to improve the SKRs, however as shown in [6], the achievable distance of CV-QKD schemes are significantly shorter compared to twin-field (TF) DV-QKD schemes [9–11]. As an illustration, the authors in [12] employed the multicore fiber with 37 cores to achieve the SKR of 105.7 Mb/s, but transmitted distance was only 7.9 km.

Post-quantum cryptography (PQC) is an advocated alternative to QKD [13–19]. The key idea behind the PQC is to design and implement the public-key protocols that are unbreakable by any quantum computer. The predominant PQC algorithms can be placed into four categories: (i) lattice-based cryptography that will be described in Sect. 12.2, (ii) code-based cryptography that will be described in Sect. 12.3, hash-based cryptography in Sect. 12.4, and multivariate cryptography in Sect. 12.5.

Yet, similar to computational security, there is no evidence that PQC algorithms are unbreachable by sophisticated quantum algorithms. As an illustration, the lattice cryptography algorithms rely on conjecture that there is no polynomial time algorithm approximating lattice problems within polynomial factors [17]. Moreover, very often lattice cryptography is based on the collision resistance hash functions, such as: $\mathbf{u} = \mathbf{A}\mathbf{x}$, with $\mathbf{x}$ the Alice private vector, $\mathbf{u}$ the Alice public vector, and $\mathbf{A}$ the $m \times n$ public matrix describing the lattice, with columns the basis vectors of the lattice. Eve needs to perform an efficient quantum matrix inversion algorithm, similar to the Harrow-Hassidim-Lloyd (HHL) algorithm [20], to determine Alice’s private vector by $\mathbf{u} = \mathbf{A}^{-1}\mathbf{x}$ and thus break the PQC protocol. The HHL algorithm is an exponential speedup over classical algorithms. To overcome the main problems of both QKD and PQC protocols, an option is to use joint QKD-PQC protocols [21]; however, even though the transmission distance of the phase-matching (PM)-TF-QKD protocol [10] can be doubled, the secret-key rate is orders of magnitude lower compared to data rates used in the state-of-the-art optical communication systems. In our recent paper [1], we have proposed to use a new strategy to overcome the above problems for QKD and PQC in a simultaneous manner. Given the low raw key rates in QKD protocols, to get the common secure sequence we propose to use the traditional QKD schemes only in the initialization stage of the proposed protocols. Contrary to conventional QKD schemes, our common secure sequence will not be a symmetric key but instead initialize the proposed cryptosystems. The key idea is to use the common secure sequence from QKD not as a secure key, but rather for: (i) a secure sequence of public keys; (ii) seeds for corresponding random “hash function” generators by Alice and Bob; (iii) parameters to initialize the protocols; etc. These secure sequences will be much shorter than the key length for one-time-pad encryption, hence the SKR of the corresponding QKD protocol is not a major concern. The hybrid QKD-PQC schemes will be described in Sect. 12.6.

12.2 Lattice-Based Cryptography

The lattice is generated by n linearly independent vectors $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^m$, where $\mathbb{R}$ is the set of real numbers, as follows:

$$L(\mathbf{b}_1, \dots, \mathbf{b}_n) = \{x_i \mathbf{b}_i \mid x_i \in \mathbb{Z}\}, m \geq n \quad (12.1)$$

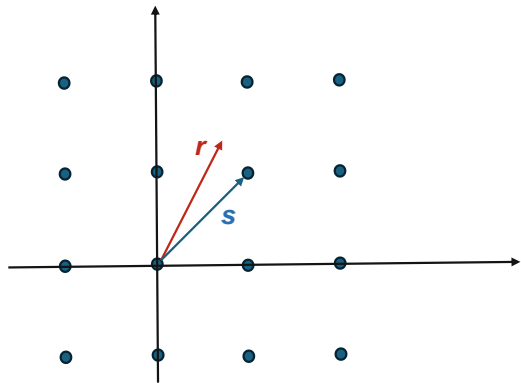
with $\mathbb{Z}$ denoting the set of integers. By writing the m -components basis vectors as columns we get the $m \times n$ matrix $\mathbf{A} = [\mathbf{b}_1 \ \mathbf{b}_2 \ \dots \ \mathbf{b}_n] \in \mathbb{R}^m \times n$ so that we can re-define the lattice as $L(\mathbf{b}_1, \dots, \mathbf{b}_n) = L(\mathbf{A}) = \{\mathbf{A}\mathbf{x} \mid \mathbf{x} \in \mathbb{Z}^n\}$.

Given the lattice there exist certain problems that are difficult to solve including:

- The *shortest vector problem* (SVP): Given the lattice described by its basis $\mathbf{A}$, determine the shortest nonzero vector in the lattice $L(\mathbf{A})$, with an example provided in Fig. 12.1.
- The *closest vector problem* (CVP): Given the lattice, described by its basis $\mathbf{A}$, and a target vector $\mathbf{t}$, not in the lattice, determine the vector $\mathbf{v}$ in the lattice closet to the target vector $\mathbf{t}$.
- *Bounded distance decoding*: Similarly to the CVP, determine the lattice point $\mathbf{s}$ closest to the received (target) vector $\mathbf{r}$ ($\mathbf{t}$).

Another relevant problem is the *shortest independent vectors problem* (SIVP), which can be formulated as follows: Given a lattice with the basis $\mathbf{A} \in \mathbb{Z}^n \times n$ determine n linearly independent vectors $\mathbf{S} = [s_1 \ s_2 \ \dots \ s_n]$ minimizing the $\|\mathbf{S}\| = \max_i \|s_i\|$ where $\|s_i\|$ is the norm of s_i . A very popular norm is the Euclidean norm, defined as $\|s_i\| = \left(\sum_j s_{ij}^2\right)^{1/2}$.

Fig. 12.1 Illustrating the shortest vector problem



12.2.1 Module Learning with Errors (LWE)-Based Cryptography

The lattice-based cryptography algorithms are related to some of the problems listed above. In learning with errors (LWE)-based cryptography, we chose the elements of the basis vectors to be randomly selected from Z_q^n , where Z_q is the set of integers per mod q . The matrix A and parameter q are made public. A simple LWE-based protocol, known as the *module learning with errors (module-LWE)*, can be described as follows. The Alice private (secret) vector $|s_A\rangle$ is related to the public vector $|p_A\rangle$ by $|p_A\rangle = A|s_A\rangle$. The generation of the matrix A and Alice public vector $|p_A\rangle$ represent the key generation step. Bob generates the private vector $\langle s_B|$ and the error vector $\langle e|$ and use them to generate the public raw vector $\langle p_B|$ and scalar, being the part of the cyphertext, c as follows: $\langle p_B| = \langle s_B|A + \langle e|$, $c = \langle s_B|p_A\rangle + e + bq/2$, where $b \in \{0,1\}$ is the bit to be encrypted. The error components e_i ($i = 1, \dots, m$) and e are chosen randomly but need to be $< q/4$. Therefore, Bob performs the encryption step to obtain the ciphertext composed of $\langle p_B|$ and c . Alice decrypts by employing her private vector $|s_A\rangle$ as follows:

$$\begin{aligned}
 b' &= \left(\frac{2}{q}\right)(c - \langle p_B|s_A\rangle) = \left(\frac{2}{q}\right) \left[\underbrace{\langle s_B|A|s_A\rangle}_{|p_A\rangle} + e + bq/2 - (\langle s_B|A + \langle e|)|s_A\rangle \right] \\
 &= \left(\frac{2}{q}\right) [\langle s_B|A|s_A\rangle + e + bq/2 - \langle s_B|A|s_A\rangle - \langle e|s_A\rangle] \\
 &= b + \frac{2}{q}(e - \langle e|s_A\rangle)
 \end{aligned} \tag{12.2}$$

Given that $(2/q)(e - \langle e|s_A\rangle)$ is a small number, behaving as the noise, then the decrypted bit b' will be equal to Bob's bit b .

We can also provide the following interpretation [22]. In key generation step, Alice creates the A-list as a length- n vector of random numbers observed mod q , with elements $A_i \in Z_q$. She also creates the error pattern of length n with components also from Z_q but smaller than q/n . She then creates the B-list as follows:

$$B_i = A_i s + e_i \text{ mod } q, \tag{12.3}$$

where s is the secret number. She publishes A- and B-lists as the public key. Bob encrypts bit b by sampling taking K -samples from B-list as follows

$$v = \sum_{j \in \text{Positions}} B_j - bq/2 \text{ mod } q, \tag{12.4}$$

where the set *Positions* denotes the locations of samples. To decrypt Alice first creates the u-number by:

$$u = \sum_{j \in \text{Positions}} A_j \bmod q, \quad (12.5)$$

and decrypts the Bob's bit by:

$$b' = \frac{2}{q} (v - sum \bmod q), \quad (12.6)$$

by rounding.

12.2.2 Ring *LWE*-Based Cryptography

For $m = n$, when the integer matrix A is the cyclic matrix:

$$A = \begin{bmatrix} a_0 & a_1 & \cdots & a_{n-2} & a_{n-1} \\ a_{n-1} & a_1 & \cdots & a_{n-3} & a_{n-2} \\ \vdots & & \ddots & \vdots & \vdots \\ a_1 & a_2 & \cdots & a_{n-1} & a_0 \end{bmatrix}, \quad (12.7)$$

we can use the polynomial description by:

$$a(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1} \bmod (x^n + 1) \quad (12.8)$$

The secret polynomial $s(x)$ and the error polynomial $e(x)$ will be respectively:

$$\begin{aligned} s(x) &= s_0 + s_1x + \cdots + s_{n-1}x^{n-1} \bmod (x^n + 1), \\ e(x) &= e_0 + e_1x + \cdots + e_{n-1}x^{n-1} \bmod (x^n + 1). \end{aligned} \quad (12.9)$$

Now, we can define the ring R as the ring of integer polynomials $\bmod (x^n + 1)$, that is $\mathbb{Z}_q[x]/(x^n + 1)$, where n is the power of 2. The secret $\mathbf{s} = (s_0 \ s_1 \ \dots \ s_{n-1})$ will be an element from the ring R . The coefficients of the polynomial $a(x)$, denoted as $\mathbf{a} = (a_0 \ a_1 \ \dots \ a_{n-1})$, are now chosen uniformly for the ring R . Finally the elements of the error polynomial $e(x)$, denoted as $\mathbf{e} = (e_0 \ e_1 \ \dots \ e_{n-1})$, are now chosen as small independent variables from the zero-mean Gaussian distribution of standard deviation αq , where $\alpha > 0$ is properly chosen small positive real number. The corresponding encryption protocol is known as the *ring LWE-based cryptography* [19]. The Alice will create the public polynomial $p(x)$ by:

$$p(x) = a(x)s(x) + e(x). \quad (12.10)$$

From an eavesdropper point for view, it is hard to solve for the secret polynomial $s(x)$ given the polynomial system:

$$\begin{aligned} a_0(x)s(x) + e_0(x) &= p_0(x) \\ a_1(x)s(x) + e_1(x) &= p_1(x) \\ &\vdots \\ a_{L-1}(x)s(x) + e_{L-1}(x) &= p_{L-1}(x) \end{aligned} \quad (12.11)$$

as error polynomials $e_i(x)$ are not known.

Another relevant protocol is the *ring LWE for key exchange*. The goal is to create the shared encryption key. Alic and Bob first agree on the shared polynomial $a(x)$, n , and q . Then Alice generates secret polynomial $s_A(x)$ and Bob generates his secret polynomial $s_B(x)$. They also generate error polynomials $e_A(x)$ and $e_B(x)$, respectively. The corresponding public polynomials are further created by:

$$\begin{aligned} p_A(x) &= a(x)s_A(x) + e_A(x), \\ p_B(x) &= a(x)s_B(x) + e_B(x). \end{aligned} \quad (12.12)$$

Alice then takes Bob's public polynomial and multiplies it with her secret polynomial to obtain:

$$\begin{aligned} k_A(x) &= p_B(x)s_A(x) = [a(x)s_B(x) + e_B(x)]s_A(x) \\ &= a(x)s_B(x)s_A(x) + e_B(x)s_A(x). \end{aligned} \quad (12.13)$$

In parallel, Bob takes Alice's public polynomial and multiplies it with his secret polynomial to obtain:

$$\begin{aligned} k_B(x) &= p_A(x)s_B(x) = [a(x)s_A(x) + e_A(x)]s_B(x) \\ &= a(x)s_A(x)s_B(x) + e_A(x)s_B(x). \end{aligned} \quad (12.14)$$

Clearly, if error polynomials are set to zero, the corresponding keys will be identical $k(x) = a(x)s_A(x)s_B(x)$. To remove the errors Alice and Bob apply the following decision rule on their keys to get the same shared encryption key:

$$\hat{k}_i = \begin{cases} 0, & (k_{A,B})_i < q/4 \\ 1, & q/4 < (k_{A,B})_i < q/2 \\ 0, & q/2 < (k_{A,B})_i < 3q/4 \\ 1, & (k_{A,B})_i > 3q/4 \end{cases} \quad (12.15)$$

12.2.3 Lattice-Based Digital Signatures

In the rest of this section, we describe the *lattice-based digital signature* procedure. The ring R is defined in terms of polynomials with coefficients from Z_q , observed per mod $(x^n + 1)$. The sender creates the matrix A of size $m \times l$, with elements from Z_q . The sender also creates the $l \times 1$ secret column vector $|s_0\rangle$, with elements from Z_q , as well as the $m \times 1$ error vector $|e_0\rangle$. The matrix A , secret vector, and error vector represent the secret key used for signing. The private key is composed of the matrix A and the private vector $|p_0\rangle$, created as $|p_0\rangle = A|s_0\rangle + |e_0\rangle$. The public key is broadcasted publicly, and it is used on destination side to verify the sender. In signing procedure, the sender creates another secret vector $|s_1\rangle$ and applies the matrix A to it to get $|w\rangle = [A|s_1\rangle]$, where we use $[\cdot]$ to denote the approximation procedure in which we keep the higher order bits only. Now the sender applies the hashing function H to the concatenation of the message M and column vector $|w\rangle$ to get $h = H(M||w)$, where we use the $||$ to denote the concatenation operation. The signed message is further created by $|z\rangle = |s_1\rangle \pm h|s_0\rangle$. The vector $|z\rangle$ needs to be a valid signature, not correlated to the secret vector $|s_0\rangle$. Finally, the sender sends the signed message to destination node, composed of M , $|z\rangle$, and h . The verifier on destination node employs the public key to verify the sender by creating the u -vector as follows:

$$|u\rangle = [A|z\rangle \mp h|p_0\rangle], \quad (12.16)$$

wherein the sign needs to be opposite of that used on the sender side. The sender is verified if the output hashing function is the same as that provided by the sender, that is:

$$v = H(M||u) \stackrel{?}{=} h. \quad (12.17)$$

What remained is to show that u -vector is indeed related to the sender's w -vector:

$$\begin{aligned} |u\rangle &= \left[A \underbrace{|z\rangle}_{|s_1\rangle \pm h|s_0\rangle} \mp h \underbrace{|p_0\rangle}_{A|s_0\rangle + |e_0\rangle} \right] = [A|s_1\rangle \pm hA|s_0\rangle \mp hA|s_0\rangle \mp h|e_0\rangle] \\ &= [A|s_1\rangle \mp h|e_0\rangle] \\ &\approx |w\rangle. \end{aligned} \quad (12.18)$$

Clearly, the approximation operation $[\cdot]$, which keeps only higher order bits, helps removing the effect of small errors.

12.2.4 Lattice-Based Hash Functions

One-way hash functions have already been introduced in Chap. 3. Here the focus is on collision resistant hash functions based on lattices [17], denoted by $f: D \rightarrow R$, which map the domain D to range set of much smaller size, that is $|R| < |D|$. Typically, the hash functions are defined as the keyed function families, wherein the family of functions $\{f_k: D \rightarrow R\}$ is specified. Given that the hash function is chosen at random, say f_k , an attacker cannot easily find a collision in f_k .

One of the first lattice based hash functions' construction that provided worst-case security guarantees, was the *Ajtai's construction* [23, 24]. This construction relies on the hardness of the n^c -approximate shortest vector problem (for $c > 0$), and it was initially presented as one-way hash functions' construction. It was later proved in [23] that this construction is actually collision resistant, which is a stronger security property. The Ajtai's hash function is defined by:

$$f_A(\mathbf{y}) = \mathbf{A}\mathbf{y} \bmod q, \mathbf{A} \in \mathbb{Z}_q^{n \times m}, \quad (12.19)$$

where $\mathbf{A}$ is selected uniformly from $\mathbb{Z}_q^{n \times m}$, and $\mathbf{y} \in \mathbb{Z}_q^m$ is the "small" input vector. Given that components of input vector are small, it is extremely difficult to perform the matrix inversion $\mathbf{A}^{-1}$. This hash functions performs the following mapping $f_A: \{0, 1, \dots, d-1\}^m \rightarrow \mathbb{Z}_q^n$, maps $(m \log d)$ bits to $(n \log q)$ bits, and to have compression properties it is required that $m > n \log q / \log d$.

The efficiency of the Ajtai's construction can be improved by replacing the random matrix $\mathbf{A}$ with block-circulant matrix [17]:

$$\mathbf{A} = [\mathbf{A}^{(1)} | \mathbf{A}^{(2)} | \dots | \mathbf{A}^{(m/n)}], \mathbf{A}^{(i)} = \begin{bmatrix} a_1^{(i)} & a_n^{(i)} & \dots & a_2^{(i)} \\ a_2^{(i)} & a_1^{(i)} & \dots & a_3^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1}^{(i)} & a_{n-2}^{(i)} & \dots & a_n^{(i)} \\ a_n^{(i)} & a_{n-1}^{(i)} & \dots & a_1^{(i)} \end{bmatrix}; i = 1, \dots, m/n \quad (12.20)$$

The i -th circulant block can be represented I matrix forms as:

$$\mathbf{A}^{(i)} = \mathbf{F}^* \mathbf{a}^{(i)} = \begin{bmatrix} a^{(i)} | \mathbf{F} \mathbf{a}^{(i)} | \mathbf{F}^2 \mathbf{a}^{(i)} | \dots | \mathbf{F}^{n-1} \mathbf{a}^{(i)} \end{bmatrix}, \mathbf{F} = \begin{bmatrix} 0 & 0 & \dots & | & \\ \ddots & & & | & -\mathbf{f} \\ & \mathbf{I} & & | & \\ & & \ddots & | & \end{bmatrix} \quad (12.21)$$

with $\mathbf{f} = [-1 \ 0 \ \dots \ 0]^T$. Now by generalizing $\mathbf{f}$ to be a column vector from Z^n , we can formulate the *ideal lattices based hash function construction* as follows [17]:

- **Parameters** of the construction: Integers m, n, d, q with $n|m$ and column vector $\mathbf{f}$ being from Z^n .
- **Key**: m/n vectors $\mathbf{a}_1, \dots, \mathbf{a}_{m/n}$ are chosen independently and uniformly at random from Z_q^n .
- **Hash function** is defined by:

$$f_A(\mathbf{y}) = [\mathbf{F}^* \mathbf{a}_1 | \mathbf{F}^* \mathbf{a}_2 | \dots | \mathbf{F}^* \mathbf{a}_{m/n}] \mathbf{y} \bmod q, \quad (12.22)$$

and performs the mapping $f_A : \{0, 1, \dots, d-1\}^m \rightarrow Z_q^n$.

This algorithm has been named ideal lattices based because for general $\mathbf{f}$ the corresponding lattices can be equivalently described as *ideals* of the ring $Z[x]/\langle f(x) \rangle$, where $f(x) = x^n + f_n x^{n-1} + \dots + f_1$ [25]. As shown in [26], this lower complexity construction has the efficiency of the one-way function. By employing the Fast Fourier transform (FFT) over the complex number, this algorithm can be computed asymptotically in $O(m)$.

By introducing the FFT in Z_q authors in [27] introduced the construction that is known as the *SWIFFT hash function*. Similarly to the discrete Fourier transform (DFT), we can describe the FFT in terms of the Vandermonde matrix but now of elements $\omega, \omega^3, \dots, \omega^{2n-1}$

$$\mathbf{W} = \left[\omega^{(2i-1)(j-1)} \right]_{i,j=1}^n, \quad (12.23)$$

wherein ω is an element of the multiplicative group Z_q^* (integers mod q) of order $2n$. By selecting q to be a prime and at the same time ensuring that $2n$ divides $q-1$, the matrix $\mathbf{W}$ becomes an invertible matrix in Z_q . Multiplication by $\mathbf{W}^{-1}$ maps a uniformly chosen $\tilde{\mathbf{a}} \in Z_q^n$ to also uniformly chosen $\mathbf{a} \in Z_q^n$. For $\mathbf{A} = [\mathbf{F}^* \mathbf{a}_1 | \mathbf{F}^* \mathbf{a}_2 | \dots | \mathbf{F}^* \mathbf{a}_{m/n}]$, the SWIFFT maps a key consisting of m/n vectors $\tilde{\mathbf{a}}_1, \tilde{\mathbf{a}}_2, \dots, \tilde{\mathbf{a}}_{m/n}$ (chosen randomly at random, and independently from Z_q^n) and an input $\mathbf{y} \in \{0, 1, \dots, d-1\}^m$ to [17]:

$$\mathbf{W} \cdot f_A(\mathbf{y}) \bmod q = \sum_{i=1}^{m/n} \mathbf{W} [\mathbf{F}^* \mathbf{a}_i] \mathbf{y}_i = \sum_{i=1}^{m/n} \tilde{\mathbf{a}}_i \odot (\mathbf{W} \mathbf{y}_i), \quad (12.24)$$

where denotes the component-wise vector product. Now we can formulate the SWIFFT hash function as follows [17]:

- **Parameters** of the construction: Integers m, n, d, q with n being power of 2, $n|m$, $2n|(q-1)$.
- **Key**: m/n vectors $\tilde{\mathbf{a}}_1, \tilde{\mathbf{a}}_2, \dots, \tilde{\mathbf{a}}_{m/n}$ are chosen independently and uniformly at random from Z_q^n .

Input: m/n vectors $y_1, \dots, y_{m/n} \in \{0, 1, \dots, d-1\}^m$. **Hash function:** $\sum_{i=1}^{m/n} \tilde{a}_i \odot (Wy_i)$

Regarding the complexity and security analysis of various lattice-based constructions discussed here an interested reader is referred to [17, 28].

12.3 Code-Based Cryptography

The code-based cryptography is based on error correction codes, with McEliece encryption scheme being proposed in 1978 [13], being the well-known PQC scheme. The McEliece cryptosystem based on quasi cyclic (QC)-LDPC coding [21, 29] is straightforward to implement and the corresponding LDPC encoders and decoders have been already implemented in FPGAs [30]. In Chap. 2, we have already provided the introduction to learn block coded and LDPC codes.

12.3.1 Modified McEliece Encryption Scheme

The modified McEliece encryption scheme is based on an (n, k) linear block code and can be described as follows [21]. Alice generates the generator matrix $\mathbf{G}$ of size $k \times n$ at random and publishes the public key $\mathbf{G}'$ determined by $\mathbf{G}' = \mathbf{S}^{-1} \mathbf{G} \mathbf{P}^{-1}$, where $\mathbf{S}$ is a $k \times k$ non-singular scrambling matrix and $\mathbf{P}$ is a $n \times n$ permutation matrix. (Both $\mathbf{S}$ and $\mathbf{P}$ are invertible matrices.) Bob will then encode the message bits $\mathbf{m}$ as follows $\mathbf{r} = \mathbf{m} \mathbf{G}' + \mathbf{e}$, where $\mathbf{e}$ is the error pattern (vector) of weight t , where t is the error correction capability of the code. Upon receiving $\mathbf{r}$, Alice will perform the transformation $\mathbf{r}' = \mathbf{r} \mathbf{P} = \mathbf{m} \mathbf{S}^{-1} \mathbf{G} + \mathbf{e} \mathbf{P}$ and next decode based on the parity-check matrix derived from $\mathbf{G}$ to obtain $\mathbf{m}' = \mathbf{m} \mathbf{S}^{-1}$. Finally, she will recover the message vector $\mathbf{m}$ by multiplication of $\mathbf{m}'$ by $\mathbf{S}$, from the right.

Given that permutation matrix is just changing the positions of the errors in the error pattern without affecting the weight, $\mathbf{e} \mathbf{P}$ is a correctable error pattern. Given that the decoding of a linear code without knowing the generator matrix $\mathbf{G}$ is NP-complete, and thus out of reach of quantum computers, the McEliece encryption scheme is secure.

Example. Let us employ the linear block code represented by the following parity-check matrix $\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix}$. Given that this is a systematic code, the generator matrix is given by $\mathbf{G} = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$. The permutation matrix is

given by $\mathbf{P} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$, while the inverse permutation matrix is

$\mathbf{P}^{-1} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$. For the scrambling matrix $\mathbf{S} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, the

corresponding inverse matrix is $\mathbf{S}^{-1} = \mathbf{S}$. The public generation matrix will be $\mathbf{G}' = \mathbf{S}^{-1} \mathbf{G} \mathbf{P}^{-1} = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix}$. The message vector $\mathbf{m} = [1 \ 0]$ for the error vector $\mathbf{e} = [0 \ 0 \ 1 \ 0 \ 0]$ get encrypted to $\mathbf{r} = \mathbf{m} \mathbf{G}' + \mathbf{e} = [1 \ 1 \ 1 \ 0 \ 1]$. The decryption starts with $\mathbf{r}' = \mathbf{r} \mathbf{P} = [0 \ 1 \ 1 \ 1 \ 1]$. After syndrome decoding we obtain $\mathbf{m}' = [0 \ 1]$. After multiplying $\mathbf{m}'$ with $\mathbf{S}$ from the right we obtain $\mathbf{m} = [1 \ 0]$.

12.3.2 Modified Niederreiter Cryptosystem

The simplification of the McEliece encryption scheme was done by Herald Niederreiter in 1986 [31], and this scheme is known as the *Niederreiter cryptosystem*. The key idea is to use the parity-check matrix $\mathbf{H}$ of the linear bloc code rather than the generator matrix, so that this scheme is technically equivalent to McEliece scheme and preserves the security. Another simplification is that the message $\mathbf{m}$ was encoded into the error pattern $\mathbf{e}$ of weight t . Here we provide a slightly modified version of the Niederreiter cryptosystem. The public key is given by the modified parity-check matrix $\mathbf{H}' = \mathbf{S}^{-1} \mathbf{H} \mathbf{P}^{-1}$, while the private key is defined by the triple $(\mathbf{S}, \mathbf{H}, \mathbf{P})$. The ciphertext is generated by $\mathbf{c} = \mathbf{H}' \mathbf{e}^T$. Upon receiving $\mathbf{c}$, Alice will perform the transformation $\mathbf{c}' = \mathbf{S} \mathbf{c} = \mathbf{S} \mathbf{S}^{-1} \mathbf{H} \mathbf{P}^{-1} \mathbf{e}^T = \mathbf{H} \mathbf{P}^{-1} \mathbf{e}^T$ and next decode based on the parity-check matrix $\mathbf{H}$ to obtain $\mathbf{e}'^T = \mathbf{P}^{-1} \mathbf{e}^T$ and will recover the error vector $\mathbf{e}$ by multiplication of $\mathbf{e}'^T$ by $\mathbf{P}$, from the left, to obtain $\mathbf{P} \mathbf{e}'^T = \mathbf{e}^T$. Given that error pattern contains the message $\mathbf{m}$, the decryption is successful.

12.3.3 QC-LDPC Coding-Based Encryption

The starting point is the QC-LPDC code with the template parity-check matrix [21]:

$$H_{QC} = \begin{bmatrix} \mathbf{I} & \mathbf{I} & \mathbf{I} & \dots & \mathbf{I} \\ \mathbf{B}^{S[0]} & \mathbf{B}^{S[1]} & \mathbf{B}^{S[2]} & \dots & \mathbf{B}^{S[c-1]} \\ \mathbf{B}^{2S[0]} & \mathbf{B}^{2S[1]} & \mathbf{B}^{2S[2]} & \dots & \mathbf{B}^{2S[c-1]} \\ \dots & \dots & \dots & \dots & \dots \\ \mathbf{B}^{(r-1)S[1]} & \mathbf{B}^{(r-1)S[1]} & \mathbf{B}^{(r-1)S[2]} & \dots & \mathbf{B}^{(r-1)S[c-1]} \end{bmatrix}, \quad (12.25)$$

where $\mathbf{I}$ and $\mathbf{B}$ are identity and permutation matrices of size $b \times b$, and integers $S[i] \in \{0, 1, \dots, b-1\}$ ($i = 0, 1, \dots, r-1$; $r < b$) are properly chosen to satisfy the girth (the largest cycle in corresponding bipartite graph representation of H_{QC}) constrains, as described in [32]. We can incorporate many QC-LDPC codes using this design. As an illustration, the column -weight-3 code of girth-10 can be designed to be a subcode of girth-8, column weight-4 code. Lower-rate code of the same girth should be a subcode of higher-rate code. This architecture allows run-time reconfiguration on codeword-by-codeword basis. Its operation has been demonstrated over a free-space optical channel in the presence of time-varying atmospheric turbulence in our recent paper [30]. Finer granulation in code rate adaptation can be implemented by shortening. For application of QC-LDPC code design in McEliece cryptosystem, we propose to generate many sets of integers $\{S[i]\}$ satisfying run-time configurability conditions and select them at random.

The modified McEliece encryption scheme based on this systematic QC-LDPC codes can be described as follows. Alice randomly chooses the number of block-columns in the template parity-check matrix of the QC-LDPC code. She generates the generator matrix $\mathbf{G}$, based on parity-check matrix and publishes the public key $\mathbf{G}'$ determined by $\mathbf{G}' = \mathbf{S}^{-1}\mathbf{G}\mathbf{P}^{-1}$. Bob will then encode the message bits $\mathbf{m}$ as follows $\mathbf{r} = \mathbf{m}\mathbf{G}' + \mathbf{e}$, where $\mathbf{e}$ is the error pattern of low weight. Upon receiving $\mathbf{r}$, Alice will perform the transformation $\mathbf{r}' = \mathbf{r}\mathbf{P} = \mathbf{s}\mathbf{S}^{-1}\mathbf{G} + \mathbf{e}\mathbf{P}$ and next decode based on the parity-check matrix to obtain $\mathbf{m}' = \mathbf{m}\mathbf{S}^{-1}$, and will recover $\mathbf{m}$ by multiplication of $\mathbf{m}'$ by $\mathbf{S}$.

The security is not affected by using the QC-LDPC code design instead of random code construction as discussed in [29], when the scrambling matrix is properly selected.

On the other hand, the modified Niederreiter cryptosystem based on QC-LDPC coding can be described as follows. Alice selects sets of integers $\{S[i]\}$ to be used in Eq. (12.25) at random and creates the corresponding parity-check matrix. The public key is given by the modified parity-check matrix $\mathbf{H}' = \mathbf{S}^{-1}\mathbf{H}_{QC}\mathbf{P}^{-1}$. The message $\mathbf{m}$ was encoded into the error pattern $\mathbf{e}$. The ciphertext is generated by $\mathbf{c} = \mathbf{H}'\mathbf{e}^T$. Upon receiving $\mathbf{c}$, Alice will perform the transformation $\mathbf{c}' = \mathbf{S}\mathbf{c} = \mathbf{S}\mathbf{S}^{-1}\mathbf{H}_{QC}\mathbf{P}^{-1}\mathbf{e}^T = \mathbf{H}_{QC}\mathbf{P}^{-1}\mathbf{e}^T$ and next decode based on the parity-check matrix $\mathbf{H}_{QC}$ to obtain $\mathbf{e}'^T = \mathbf{P}^{-1}\mathbf{e}^T$ and will recover the error vector $\mathbf{e}$ by multiplication of $\mathbf{e}'^T$ by $\mathbf{P}$, from the left, to obtain $\mathbf{P}\mathbf{e}'^T = \mathbf{e}^T$. Given that error pattern contains the message $\mathbf{m}$, the decryption has been successfully performed.

12.4 Hybrid QKD-PQC Cryptography

In our recent paper [21], we proposed joint QKD-post-quantum encryption concept that is applicable to any QKD scheme, which will be described in this section.

12.4.1 Hybrid PQC-DV-QKD Encryption

In the joint QKD-PQC cryptosystem for DV-QKD subsystem, the reverse reconciliation is used. The QKD subsystem is used for raw key transmission. Once the sifting procedure and quantum bit-error rate (QBER) estimation are completed, as shown in Fig. 12.2, Bob employs an (N, K) LDPC code with corresponding $(N-K) \times N$ parity-check matrix $\mathbf{H}$ to generate the syndrome vector $\mathbf{p} = \mathbf{x}\mathbf{H}^T$ with $\mathbf{x}$ being the Bob's sifting key.

In conventional information reconciliation, Bob would transmit the syndrome vector over an (error-free) authenticated public channel to which Eve has access. In our proposed joint encryption scheme, we will encrypt the syndrome vector by employing a properly chosen post-quantum cryptography algorithm. Unfortunately, similarly to computational cryptography, the PQC is based on unproven assumptions. Namely, some of the QPC algorithms might be broken in future by developing more sophisticated quantum algorithms. Further, the complexity of decryption algorithm in PQC dictates the number of secure bits, which is insufficient for perfect security, such as one-time pad. By using the PQC algorithms only to protect the transmission of syndrome vector of length $N-K$, which for high-rate LDPC codes is much shorter than the codeword length n used in PQC, we can achieve the perfect security. Therefore, a key idea of our proposal is to reduce leakage of information during the information reconciliation. On the other hand, by employing the QKD subsystem for raw-key transmission, we can identify the presence of the Eve. By limiting information leakage due to error correction, we can significantly extend the transmission distance. To illustrate, the secret fraction r for decoy-state-BB84 protocol can be represented as follows [1, 33]:

$$r = p_1^{(Z)} \left[1 - h_2(e^{(X)}) \right] - p_\mu^{(Z)} f_e h_2(e_\mu^{(Z)}), \quad (12.26)$$

where we used the subscript 1 to denote the single-photon pulses and μ to denote the pulse with the mean photon number μ . In (12.26) $p^{(Z)}$ denotes the probability of

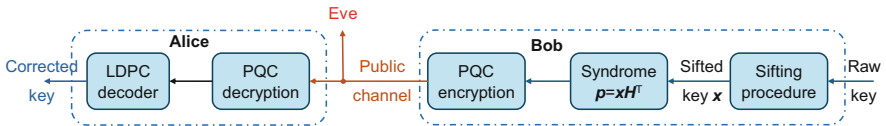


Fig. 12.2 PQC-based information reconciliation illustration

declaring a successful result (“the gain”) when Alice sent a single-photon and Bob detected it in the Z-basis, f_e denotes the error correction inefficiency ($f_e \geq 1$), $e^{(X)}$ [$e^{(Z)}$] denotes the QBER in the X-basis (Z-basis), and $h_2(x)$ is the binary entropy function. The second term $p^{(Z)}h_2[e^{(X)}]$ corresponds to the amount of information Eve was able to learn during the raw key transmission. The third term $p^{(Z)}f_e h_2[e^{(Z)}]$ denotes the amount of information revealed during the information reconciliation stage, typically related to the parity-check bits transmitted over an authenticated (noiseless) public channel. Now by transmitting the parity bits using the PQC, with number of parity bits lower than the number of security bits in PQC, the last term can be eliminated, which results in significant improvement in transmission distance. This is particularly true when the second term is close to the first term, which corresponds to the high attenuation regime.

The quantum algorithms to be developed (not yet known), capable of breaking the PQC algorithms will have certain complexity expressed in terms of number of operations L . By ensuring that the number of parity bits $N-K$ is shorter than the number of secure PQC bits $\log_2 L$, the proposed cryptographic scheme will be secure. Evidently, the proposed cryptographic scheme exploits the complexity of corresponding quantum algorithms used to break the PQC protocols. On the other hand, the conventional QKD algorithms assume that Eve is all-powerful and ignore the complexity of different quantum attacks. Therefore, the proposed cryptographic scheme belongs to the class of cryptographic schemes in which Eve is not omnipotent.

12.4.2 Spatially Coupled LDPC Coding-Based McEliece Encryption Subsystem

By using this QC-LDPC code as a template design, we create a spatially coupled (SC)-LDPC code that is depicted in Fig. 12.3, which builds on the QC-LDOC code design provided by Eq. (12.25). The codeword length of this SC-LDPC code will be $b \times (l \times c - m \times (l-1))$, where l is the number of coupled template QC-LDPC codes

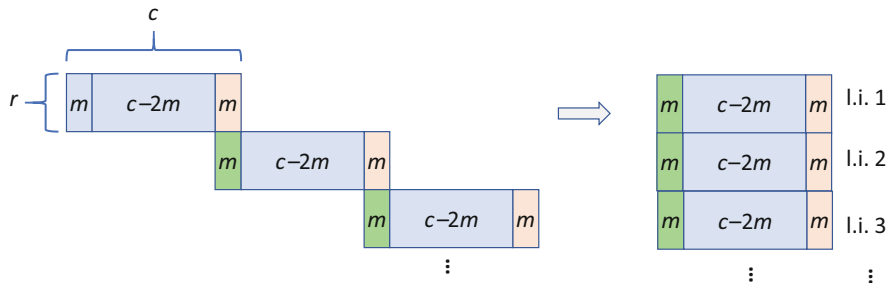


Fig. 12.3 Information reconciliation employing SC-LDPC coding based PQC

and m is the coupling length expressed in terms of number of blocks. Because there are $r \times c \times l$ non-empty sub-matrices in the parity-check matrix of the SC-LDPC code, we can introduce the layer index (l.i.) and reduce the memory requirements as illustrated in Fig. 12.3 (right). In such a way, we do not need to memorize the all-zeros submatrices. For full-rank parity-check matrix of the template QC-LDPC code (12.25), the code rate of SC-LDPC code will be simply:

$$R = 1 - \frac{rl}{lc - m(l-1)}, \quad (12.27)$$

Therefore, for fixed l , by increasing the coupling length m we can reduce the code rate and thus improve the error-correction capability of the code. For FPGA implementation of decoders for SC-LDPC codes, an interested reader is referred to our previous paper [34]. To adjust for error correction strength, depending on time-varying channel conditions, we can adapt both the template QC-LDPC code and parameters of corresponding SC-LDPC code. For application in McEliece and Niederreiter crypto-subsystems, we propose to randomly select parameters of both QC- and SC-LDPC code designs.

Regarding the McEliece encryption subsystem, Alice randomly chooses the number of block-columns in template parity-check matrix of corresponding QC-LDPC code as well as the coupling length m and number of template QC-LDPC codes l to be used in McEliece encryption scheme. She generates the generator matrix G based on the parity-check matrix from Fig. 12.3 and publishes the public key G' determined by $G' = S^{-1}GP'^{-1}$, where S is the non-singular scrambling matrix and P' is the permutation matrix. Bob will then encode the parity bits (syndrome vector) p as follows $r = pG' + e$, where e is the error pattern (vector) of low weight. Upon receiving r Alice will perform the transformation $r' = rP' = pS^{-1}G + eP'$, followed by decoding based on the parity-check matrix to obtain $p' = pS^{-1}$, and will recover p by multiplication of p' by S . Alice will further use this parity bits p to perform the error reconciliation and get corrected key u .

Regarding the Niederreiter encryption subsystem, Alice randomly chooses the number of block-columns in template parity-check matrix of corresponding QC-LDPC code as well as the coupling length m and number of template QC-LDPC codes l to be used in the encryption scheme, according to Fig. 12.3, to get the spatially coupled parity-check matrix H_{SC} , which is used as her private key. The public key is given by the modified parity-check matrix $H' = S^{-1}H_{SC}P'^{-1}$, permutation matrix. Bob will then encode the parity bits (syndrome vector) p into the error pattern e . The ciphertext is generated by $c = H'e^T$. Upon receiving c , Alice will perform the transformation $c' = Sc = SS^{-1}H_{SC}P'^{-1}e^T = H_{SC}P'^{-1}e^T$ and next decode based on the parity-check matrix H_{QC} to obtain $e'^T = P'^{-1}e^T$ and will recover the error vector e by multiplication of e'^T by P' , from the left, to obtain $Pe'^T = e^T$. Given that error pattern contains the syndrome vector p , she will use it to perform the error reconciliation and get corrected key u .

12.4.3 Hybrid Twin Field (TF)-QKD-PQC Encryption

The TF-QKD scheme with information reconciliation based on McEliece cryptosystem, proposed in Ref. [21], is depicted in Fig. 12.4. The stabilized CW lasers of low linewidth are used on Alice and Bob sides to generate the global phase stabilized optical pulses with the help of amplitude modulator. Alice and Bob choose the random phases $\phi_A \in [0, 2\pi]$ and $\phi_B \in [0, 2\pi]$, respectively, with corresponding phase modulators. The random phase difference between Alice and Bob are discretized so that:

$$\phi_{A,B} \in \Delta\phi_{k_{A,B}} = \left[\frac{2\pi}{M} k_{A,B}, \frac{2\pi}{M} (k_{A,B} + 1) \right), \quad (12.28)$$

wherein the phase-bin $\Delta\phi_k$ is discretized by $k_{A,B} \in \{0, 1, \dots, M-1\}$. Alice (Bob) then randomly selects whether to use Z-basis or X-basis. When Z-basis is selected, the phase-randomized coherent state is sent with intensity either μ or 0. When X basis encoding is selected, Alice and Bob employ corresponding phase and amplitude modulators to randomly select 0 ($\pi/2$) and π ($3\pi/2$) representing logic bits 0 and 1, and such phase-encoded pulses are sent with randomly selected intensities. The corresponding quantum states, generated by Alice and Bob, are sent towards Charlie over the ultra-low-loss fiber link. The polarization controllers (PCs) ensure that Alice and Bob's pulses have the same polarization. Charlie performs the Bell state measurements (BSMs) and announces the results. Alice and Bob then disclose their phase information, that is $k_{A,B}$ and intensities, and these are used for parameter estimation. They keep the information related to Z-basis confidential to Charlie, and these data are used for raw key. Alice and Bob then perform McEliece encryption-based information reconciliation described below, followed by the privacy amplification, to get the common secure key.

Given that the McEliece cryptosystem [13] based on QC-LDPC coding is straightforward to implement as shown in [21, 29], while the corresponding LDPC encoders and decoders have been already implemented in FPGAs [30], it represents an excellent candidate to use for transmission of parity bits in TF-QKD scheme. In

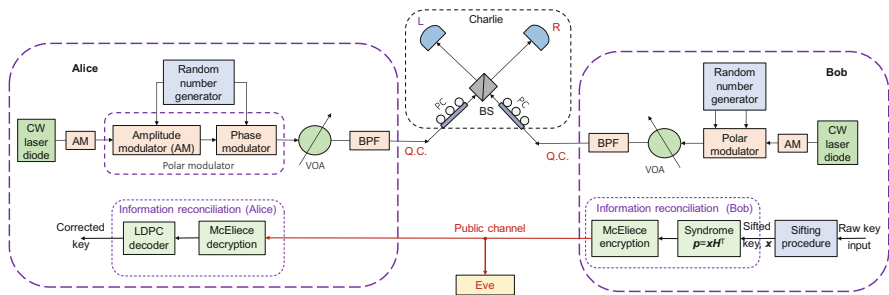


Fig. 12.4 The hybrid TF-QKD-McEliece encryption system. BS beam splitter, PC polarization controller, BPF bandpass filter, VOA variable optical attenuator

particular, the rate-adaptive spatially coupled LDPC code derived from an QC-LDPC code, and introduced in previous section, is very flexible for use in both information reconciliation and McEliece cryptosystem to encrypt the parity bits. In reverse reconciliation, based on channel conditions Bob selects the block-columns in template QC-LDPC code, coupling length for spatial LDPC coding design, and the number of spatially coupling blocks and provides to Alice the details of the spatially coupling (N, K) LDPC code design. Bob further encodes the information bits $\mathbf{u}$ obtained during sifting procedure by employing the selected spatially coupling LDPC code to get the parity bits (syndrome) $\mathbf{p}$. The McEliece encryption subsystem is the applied as described in previous section, and at the end of this procedure Alice is able to get secure key identical Bob's one.

The privacy amplification is further performed to distill from the corrected key a smaller set of bits whose correlation with Eve's string falls below the desired threshold, through the use of the universal hash functions. Assuming that Eve employs the quantum information set decoding (QISD) attack [35], the number of parity bits $N-K$ to be encrypted by (n, k) LDPC coding based McEliece encryption scheme is upper bounded by:

$$N - K \leq \log_2 \left[n^2 \sqrt{\binom{n}{k} / \left(0.29 \binom{n-t}{k} \right)} \right], \quad (12.29)$$

where t is the maximum number of errors that can be corrected by the LDPC code used in McEliece encryption subsystem. For high-rate QC- and spatially coupled LDPC codes used in information reconciliation this condition is much less stringent compared to using McEliece encryption to protect the information sequence instead.

In Fig. 12.5, we provide comparisons of the proposed joint TF-QKD-McEliece encryption scheme against the corresponding QKD subsystems employing phase-matching (PM) TF-QKD protocol introduced in [10], the MDI-QKD protocol [3], and decoy-state-based BB84 protocol [1]. The system parameters are selected as follows: the detector efficiency $\eta_d = 0.25$, reconciliation inefficiency $f_e = 1.15$, the dark count rate $p_d = 8 \times 10^{-8}$, misalignment error $e_d = 1.5\%$, and the number of phase slices for PM TF-QKD is set to $M = 16$. Regarding the transmission medium, it is assumed that recently reported ultra-low-loss fiber of attenuation 0.1419 dB/km (at 1560 nm) is used [36]. Both PM TF-QKD and joint TF-QKD-McEliece encryption schemes outperform the decoy-state BB84 protocol for distances larger than 162 km, while simultaneously outperforming MDI-QKD protocol for all distances. The PM TF-QKD protocol can achieve the maximum distance of 623 km. The proposed joint TF-QKD-McEliece encryption scheme, under QISD attack, is able to achieve the distance of even 1127 km thus significantly outperforming all other schemes. As expected, the improvement at higher normalized SKRs, such as 10^{-8} , is moderate (122 km). To improve the SKR further, we propose to employ parallel joint TF-QKD-McEliece encryption systems by employing multiple photon degrees of freedom including polarization, wavelength, OAM, and spatial modes, and similar fashion as it was done in [37].

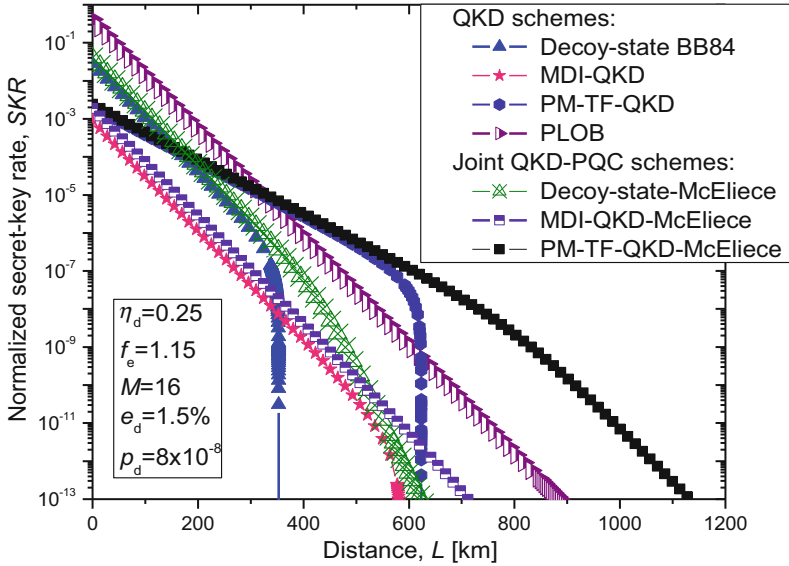


Fig. 12.5 The hybrid QKD-McEliece encryption schemes under QISD attack against decoy-state, MDI-QKD, and PM-TF-QKD in terms of SKR vs. distance

To determine what is the maximum possible transmission distance using this scheme, in Fig. 12.6 we study SKR vs. transmission distance of joint MP-TF-QKD-McEliece cryptosystem, under QISD attack, for different detector efficiencies, assuming $M = 16$, $f_e = 1.05$, and $p_d = 10^{-8}$. Clearly, for detector efficiency 0.5 the transmission distance can be extended to 1238 km. For almost ideal system parameters ($\eta_d = 0.95$, $f_e = 1.01$, and $p_d = 10^{-9}$), the maximum possible transmission distance for normalized SKR of 10^{-13} is 1355 km.

12.5 Hash-Based Cryptography

The security of cryptographic hash-functions is well established. The quantum computers cannot break the hash-functions. Given that hash-based signatures rely on difficulty of finding collision, the security of these schemes is well established compared to other PQC schemes that rely on corresponding mathematical problems hardness. The idea of hash-based signatures is old, it was proposed by Lamport in 1979 [38] to construct one-time signatures, and expanded by Merkle [39]. A good hash-function is needed for Lamport's one-time signatures, such as SHA3-512. Hash functions H map long strings $\{0,1\}^*$ to fixed-length strings $\{0,1\}^n$, that is $H: \{0,1\}^* \rightarrow \{0,1\}^n$. The cryptographic hash-functions must satisfy the following three conditions:

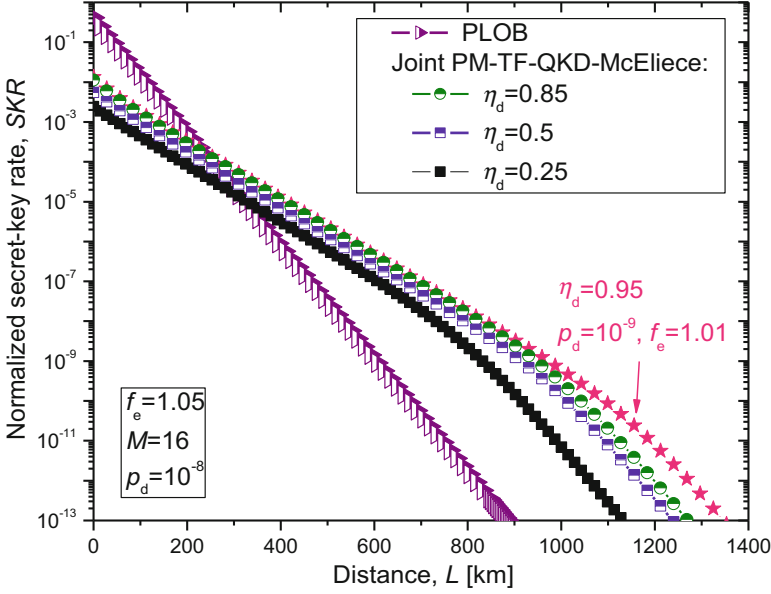


Fig. 12.6 SKR vs. distance of proposed joint QKD-McEliece cryptosystem under QISD attack for different detector efficiencies

- They are computationally *preimage resistant*, that is, the hash-function is one-way function.
- There are computationally *second image resistant*, that is, given x , $H(x)$ we cannot find $x' \neq x$ with $H(x') = H(x)$.
- There are *collision resistant*, that is, we cannot find $x' \neq x$ with $H(x') = H(x)$.

Given that Shor's factorization algorithm is not applicable here, we need to apply the Gover's search algorithm, which provides only square root improvement over the corresponding classical search algorithm.

Let us briefly describe the Lamport's one-time signature scheme for messages of length L . We assume that hash function H maps $\{0,1\}^m$ to $\{0,1\}^n$. We generate the secret key (SK) at random from $\{0,1\}^m$, that is:

$$SK = \begin{bmatrix} x_{1,0} & x_{2,0} & \cdots & x_{L-1,0} & x_{L,0} \\ x_{1,1} & x_{2,1} & \cdots & x_{L-1,1} & x_{L,1} \end{bmatrix}.$$

By applying the hash function to each symbol in SK , that is $y_{i,j} = H(x_{i,j})$, we obtain the public key (PK):

$$PK = \begin{bmatrix} y_{1,0} & y_{2,0} & \cdots & y_{L-1,0} & y_{L,0} \\ y_{1,1} & y_{2,1} & \cdots & y_{L-1,1} & y_{L,1} \end{bmatrix}.$$

To sign the message $\mathbf{m} = m_1 \dots m_L$, we simply pick the entries from the SK and send them to other party. Therefore, the signature will be $\mathbf{s} = (x_{1,m_1}, x_{2,m_2}, \dots, x_{L,m_L})$. To verify the signature $\mathbf{s} = (s_1, \dots, s_L)$ on message $\mathbf{m}$, we simply apply the hash function on $\mathbf{s}$ and compare against $\mathbf{y}$, that is $H(s_i) \stackrel{?}{=} y_{i,m_i}; i = 1, \dots, L$.

There are two major issues related to this signature scheme. The first one is disclosure of about 50% of the private key limiting the application of SK and PK pair to just a single use. The second one is the size of the signature is too large. In next subsection, we describe the Winternitz one-time signature scheme [40], which is believed to be the quantum robust.

12.5.1 Winternitz One-Time Signature (W-OTS) Scheme

Let us observe the hash function: $H: \{0,1\}^n \rightarrow \{0,1\}^n$. We use $H^l(x)$ to denote that the hash function is applied l -times, that is: $H^l(x) = H(H^{l-1}(x)) = \underbrace{H(H(\dots H(x)))}_{l\text{-times}}$.

This scheme is parametrized by the security parameter n , binary message length a , and the Winternitz integer parameter $w \geq 2$, specifying the time-memory tradeoff. We the message length L_1 and check-sum length L_2 as follows:

$$L_1 = \left\lceil \frac{a}{\log_2 w} \right\rceil, L_2 = \left\lceil \frac{\log_2 [L_1(w-1)]}{\log_2 w} \right\rceil + 1; L = L_1 + L_2 \quad (12.30)$$

The secret key (SK) is generated by taking uniformly at random L values from $\{0,1\}^n$ to get $\mathbf{SK} = (s_1, \dots, s_L)$. The public key (PK) is then obtained by applying the hash function $w-1$ times on each component of the SK to obtain:

$$\mathbf{PK} = (p_1, p_2, \dots, p_L), p_l = H^{w-1}(s_l); l = 1, \dots, L \quad (12.31)$$

For the input message $\mathbf{x} \in \{0,1\}^a$ we first represent it in the base- w with L_1 digits by $\mathbf{m} = (m_1, \dots, m_{L_1})$. We then compute the checksum with L_2 digits as follows:

$$C(\mathbf{m}) = \sum_{i=1}^{L_1} (w-1-m_i), \quad (12.32)$$

and create the $\mathbf{b}$ -vector, which is the concatenation of $\mathbf{m}$ and $C(\mathbf{m})$:

$$\mathbf{b}(\mathbf{m}) = \mathbf{m} \parallel C(\mathbf{m}) = \left(\underbrace{b_1, \dots, b_{L_1}}_{\mathbf{m}}, \underbrace{b_{L_1+1}, \dots, b_{L_1+L_2}}_{C(\mathbf{m})} \right). \quad (12.33)$$

The signature vector σ is now created with the help of the hash function H by:

$$\sigma = (H^{b_1}(s_1), \dots, H^{b_{L_1+L_2}}(s_{L_1+L_2})). \quad (12.34)$$

The $\mathbf{b}$ -vector and signature are shared with Bob.

Based on signature vector, $\mathbf{b}$ -vector, and public key $\mathbf{PK}$ Bob computes the estimated public key by:

$$\hat{\mathbf{p}} = (H^{w-1-b_1}(\sigma_1), \dots, H^{w-1-b_{L_1+L_2}}(\sigma_{L_1+L_2})), \quad (12.35)$$

end verifies if the estimated key $\hat{\mathbf{p}}$ agrees with Alice's public key $\mathbf{PK}$. If it agrees Bob accepts the signature as the Alice's signature.

12.5.2 Many-Times Signatures: Merkle Signature Scheme (MMS)

The main problem with OTS schemes is that a full-sized pair of secret and public keys must be distributed for every single message to be transmitted over the public channel. To solve this problem, Merkle proposed in 1989 an extension of OTS schemes to many-times signature scheme [40, 41], and it is based on the hash compression function $H: \{0,1\}^* \rightarrow \{0,1\}^n$. The key idea of the MMS is to use as many OTS schemes as needed, but with only one required public key. Namely, multiple public keys, required for multiple OTS schemes, are combined through the hash (Merkle) tree.

Let assume that the MSS is based on 2^L-1 instances of the W-OTS schemes (described in previous section), denoted symbolically as:

$$W-OTS_l = (\mathbf{SK}, \mathbf{PK})_l = (s_l, \mathbf{p}_l); l = 0, 1, \dots, 2^L - 1 \quad (12.36)$$

To create one public key, out of 2^L OTS public keys, Alice creates the hash (Merkle) tree, which for number of levels being $L = 3$ is provided in Fig. 12.7.

We index tree nodes, known as vertices, using two indices $v_l[k]$ with index $l = 0, 1, \dots, L$ denoting the index level and index $k = 0, 1, \dots, 2^{L-l}-1$ denoting the position of the vertex within the level. The level-0 vertices, also known as the *leaf vertices*, are obtained by applying the hash function on corresponding W-OTS $_l$ public keys $\mathbf{p}_l$ ($l = 0, 1, \dots, 2^L-1$). By moving in direction from bottom vertices towards top vertex, the contents of vertices on a higher layer are obtained by applying the hash function on the concatenation of two neighboring (even, odd) inputs from immediate lower layer, that is:

$$v_l[k] = H(v_{l-1}[2k] \parallel v_{l-1}[2k+1]); l = 1, \dots, L; k = 0, 1, \dots, 2^{L-l}-1 \quad (12.37)$$

To illustrate, the content of vertex $v_1[3]$ is obtained as:

Levels

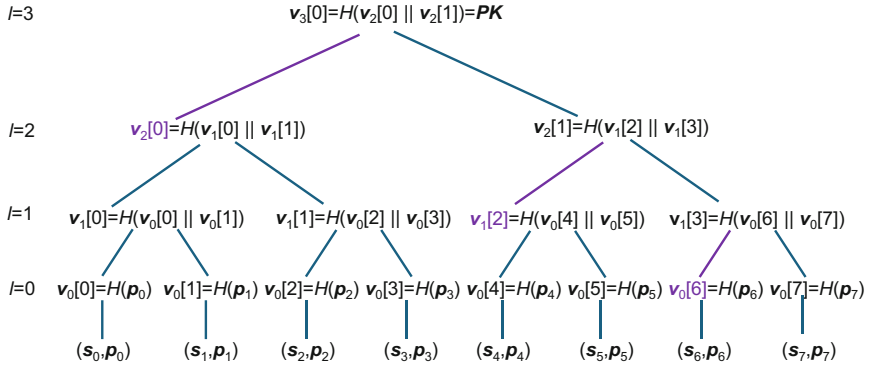


Fig. 12.7 Merkle (hash) tree with the number of levels being $L = 3$

$$v_1[3] = H(v_0[6] || v_0[7]) = H(H(p_6) || H(p_7)).$$

In another illustrative example, the top vertex content is obtained by:

$$v_1[3] = H(v_2[0] || v_2[1]) = H(H(v_1[0] || v_1[1]) || H(v_1[2] || v_1[3])) = PK$$

and represents the Alice public key, shared with everybody in the network.

To illustrate the *signature* generation process, let's assume that Alice has already utilized all private (secret) keys for the leaf vertices but the last one s_7 . To send the next message m , Alice will sign it with the help of secret OTS key s_7 by

$$\sigma_3 = \text{Signature}_{s_7}(m).$$

In traditional W-OTS scheme Alice would share the triple with Bob:

$$(m, p_7, \sigma_7)$$

with the first component being the message, the second public key, and the last one signature. Unfortunately, this triple is insufficient for Bob to verify if the message originated from Alice. Namely, Eve could pretend to be Alice and send a similar triple. Bob needs some help, which is known as the *authentication path*. In this example, the authentication path will be $(v_0[6], v_1[2], v_2[0])$, which is highlighted in Fig. 12.7. No Bob has sufficient information to verify if the signed message originated from Alice by computing the following chain of operations:

$$\begin{aligned}
\hat{\mathbf{v}}_0[7] &= H(\mathbf{p}_7) \text{ at level 0} \\
\hat{\mathbf{v}}_1[3] &= H\left(\mathbf{v}_0[6] \parallel \hat{\mathbf{v}}_0[7]\right) \text{ at level 1} \\
\hat{\mathbf{v}}_2[1] &= H\left(\mathbf{v}_1[2] \parallel \hat{\mathbf{v}}_1[3]\right) \text{ at level 2} \\
\mathbf{v}_3[0] &= H\left(\mathbf{v}_2[0] \parallel \hat{\mathbf{v}}_2[1]\right) = \hat{\mathbf{p}} \text{ at level 3}
\end{aligned}$$

If Bob's evaluated top vertex public key $\hat{\mathbf{p}}$ agrees with one already in public domain **PK**, Bob accepts the Alice signature.

12.6 Multivariate Cryptography

Multivariate cryptography is based on the hardness of finding the solution of the system of multivariate equations. The very popular multivariate system is the *multivariate quadratic* (MQ) system of equations [42–44]. The MQ problem can be formulated as follows: given the coefficients a_{ij} , b_{li} , y_l determine the solution $(x_1, \dots, x_n)$ of the following system of equations:

$$\begin{aligned}
&\sum_{i,j} a_{ij} x_i x_j + \sum_i b_{li} x_i = y_l, \\
&\vdots \\
&\sum_{i,j} a_{mij} x_i x_j + \sum_i b_{mi} x_i = y_m.
\end{aligned} \tag{12.38}$$

More compact description of the MQ problem is provided in [44]: For positive integers m, n , and a prime q , let $F(x_1, \dots, x_n)$ be a system of m polynomials over the finite field F_q in n variables $(x_1, \dots, x_n) = \mathbf{x}$. Find $\mathbf{z} \in F_q^n$ such that $F(\mathbf{z}) = \mathbf{0}$. It has been shown by Garey and Johnson in [45] that this problem is NP-hard.

Another relevant problem is the *constrained MQ* (CMQ) problem: For positive integers m, n, L , and a prime q , let $F(x_1, \dots, x_n)$ be a system of m polynomials over the finite field F_q in n variables $(x_1, \dots, x_n) = \mathbf{x}$. Find $\mathbf{z} \in \mathbb{Z}^n$ such that $F(\mathbf{z}) = \mathbf{0}$ and the i -th component $z_i \in [-L/2, L/2]$. Clearly, the solution of the CMQ problem is also a solution to the MQ problem.

Now we are in position to describe the *multivariate public key cryptography* (MPKC) scheme [46]. For integers m, n let $F(x_1, \dots, x_n)$ be a polynomial ring in $(x_1, \dots, x_n)$ over a finite field F . The *public key* is composed of:

- Easy-to-invert quadratic map $F: F^n \rightarrow F^m$ and
- Two invertible linear maps $S: F^n \rightarrow F^n$ and $T: F^m \rightarrow F^m$

The *public key* is defined by quadratic composition map $P = T \circ F \circ S: F^n \rightarrow F^m$. The corresponding *multivariate cryptographic scheme* is provided in Fig. 12.8. Alice uses the public key map P to encrypt the message. On the other hand, Bob

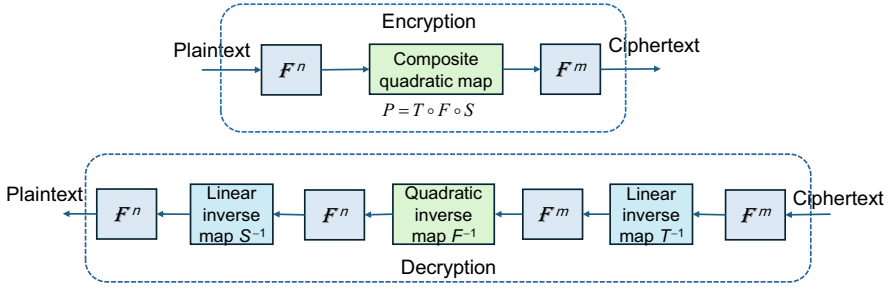


Fig. 12.8 Illustration of the multivariate cryptographic scheme

employs the private key and applies the inverse maps T^{-1} , F^{-1} , and S^{-1} to decrypt the Alice message.

The security of the MPKC schemes relies not only on the hardness of the MQ problem but also on the isomorphism of polynomials (IP) problem, which requires to recover transformation between two sets of multivariate polynomials [43], and Refs. [47, 48] have already shown that certain MPKC schemes are insecure. To solve this problem, authors in [42] proposed to use a random instance of the MQ function rather than the composite function. By going back to Eq. (12.38), in this method the coefficients a_{lij} , b_{li} , y_l are common to all users. The private key is the input to the MQ function $\mathbf{x} = (x_1, \dots, x_n)$, while the public key is the output of the MQ function $\mathbf{y} = (y_1, \dots, y_m)$. Security is now guaranteed by the intractability of the MQ problem.

12.7 Concluding Remarks

This chapter has been focused on the post-quantum cryptography (PQC) concepts. In Sect. 12.1, the basic ideas and problems of the PQC are introduced. Section 12.2 is related to the lattice-based cryptography concepts. The following concepts from the lattice-based cryptography have been described: module learning with errors (LWE)-based cryptography in subsection 12.2.1, ring LWE-based cryptography in subsection 12.2.2, lattice-based digital signatures in Sect. 12.2.3, and lattice-based hash functions in Sect. 12.2.4. The code-based cryptography concepts have been described in Sect. 12.3. The following concepts from the code-based cryptography have been described: McEliece encryption scheme (Sect. 12.3.1), Niederreiter cryptosystem (Sect. 12.3.2), and quasi-cyclic (QC)-LDPC coding-based encryption (subsection 12.3.3). In Sect. 12.4, the hybrid QKD-PQC concepts have been described. In these hybrid schemes, the QKD and PQC have been operated in a cooperative fashion. In the hybrid cryptography scheme, the raw key transmission has been performed over QKD subsystem, while information reconciliation over the PQC subsystem. The following topics have been described: hybrid PQC-QKD encryption (Sect. 12.4.1), spatially coupled LDPC coding-based McEliece encryption subsystem (Sect. 12.4.2), and hybrid twin field (TF)-QKD-PQC encryption

(Sect. 12.4.3). The hash-based cryptography has been described in Sect. 12.5. After introducing the basic concepts form the hash-based cryptography including Lamport's one-time signature scheme, two relevant schemes have been described: Winternitz one-time signature (W-OTS) scheme in Sect. 12.5.1 and many-times signatures scheme known as the Merkle signature scheme in Sect. 12.5.2. In Sect. 12.6, relevant multivariate cryptography concepts have been described.

12.8 Problems

- Let the lattice be defined by the set of basis vectors over Z_7^3 : $\{[1\ 0\ 2]^T, [4\ 5\ 0]^T, [6\ 5\ 4]^T\}$.
 - Assuming that the error vector is not used to generate the public key, for the public key $[1\ 2\ 5]^T$ determines the secret (private) key.
 - Let us now observe the same problem but when the error vector is applied. For the public vector $[0\ 6\ 5]^T$ determines the secret key.
 - Solve the problem in (b) by using the syndrome decoding.
- By using the same set of basis vectors from Problem 1 assuming that Alice and Bob's public keys are given by $[0\ 6\ 5]^T$ and $[3\ 3\ 3]^T$, respectively, represent your name using ASCII format, in hexadecimal form. Concatenate the corresponding HEX numbers and convert them to mod 7 sequence of numbers. By acting as Bob encrypt your name and surname and send it to Alice. Describe Alice's decryption procedure by using module LWE-cryptography.
- Implement the LWE-cryptographic scheme in either Matlab or Python, which is operational for arbitrary prime numbers. To verify it, use Problem 2.
- By using A/B -lists interpretation of LWE-cryptographic scheme implement it in either Matlab or Python and compare its efficiency against the implementation in Problem 3.
- Implement in Matlab or Python the ring-LWE cryptographic scheme and discuss its efficiency and security against those in Problems 3 and 4.
- An (n,k) linear block code is described by the following parity-check matrix:

$$H = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

- Determine the code parameters of this code: codeword length, number of information bits, code rate, overhead, minimum distance, error correction capability, and error detection capability.

- (b) Represent the $\mathbf{H}$ -matrix in systematic form and determine the generator matrix $\mathbf{G}$ of the corresponding systematic code. Provide the corresponding shift register based encoding circuit of this systematic code.
 - (c) Implement syndrome decoding algorithm for this code.
7. Implement McEliece encryption scheme by employing linear block code from Problem 6. Discuss its complexity and security.
 8. Implement Niederreiter cryptosystem by employing linear block code from Problem 6. Compare its complexity and security against the McEliece encryption scheme from Problem 7.
 9. This code is an extended code of the linear block code provided in Problem 6.
 - (a) Provide the corresponding parity-check matrix and put it systematic form. Determine the generator matrix.
 - (b) Determine the code parameters of the extended code: codeword length, number of information bits, code rate, overhead, minimum distance, error correction capability, and error detection capability.
 - (c) Implement syndrome decoding algorithm for this code.
 10. Implement McEliece encryption scheme by employing linear block code from Problem 9. Compare its complexity and security against McEliece encryption scheme from Problem 7.
 11. Implement Niederreiter cryptosystem by employing linear block code from Problem 9. Compare its complexity and security against the Niederreiter cryptosystem from Problem 8.
 12. By using the QC-LDPC code design from Eq. (12.25) for the permutation matrix $\mathbf{B}$ of size 2311×2311 , the set of rows $\{1,3,6\}$ design the LDPC code of girth-10 and column-weight 3. Implement McEliece encryption scheme by employing this LDPC code. Evaluate its complexity and security.
 13. By using the QC-LDPC code from Problem 12, implement the corresponding Niederreiter cryptosystem. Compare its complexity and security against the McEliece encryption scheme from Problem 12.
 14. We are interested in using the Lamport one-time signature (OTS) scheme to sign 10 MB PDF document. If each bit of the message is signed by this OTS scheme, what will be the length of the public key? If the SHA-256 is applied first to the message, what would be the length of corresponding public key?
 15. Represent your name and the date of birth using the ASCII format, in binary form. Concatenate the corresponding binary numbers. Determine the W-OTS public key using $n = 11$ bits and $w = 3$ for the hash function $H(x) = x^2 + 1 \bmod 97$ and assuming that the private key is generated randomly. Use the ASCII representation of your name, surname, and the date for birth as the message. What is the required length of the private key? Determine the signature of the message. Apply the verification procedure to see if the signature is a valid one.
 16. We want to generate the MSS signature for the Merkle tree with $L = 3$ levels. Let us use the following hash function $H(x) = x^2 + x + 1 \bmod 491$.

- (a) Randomly generate the secret keys and hash them to get the public keys.
 - (b) Compute the MSS public key.
 - (c) Represent your name, surname, and the date of birth using ASCII representation (do not concatenate them) and consider them as three messages. Generate the corresponding MSS signatures.
 - (d) Describe the verification process.
17. Describe how CV-QKD can be combined with PQC schemes. Describe how to determine the secret key rates for this hybrid scheme.

References

1. Djordjevic, I. B. (2021). QKD-enhanced cybersecurity protocols. *IEEE Photonics Journal*, 13(2), 1–8, Art no. 7600208. <https://doi.org/10.1109/JPHOT.2021.3069510>
2. Lo, H.-K., Ma, X., & Chen, K. (2005). Decoy state quantum key distribution. *Physical Review Letters*, 94, 230504.
3. Lo, H.-K., Curty, M., & Qi, B. (2012). Measurement-device-independent quantum key distribution. *Physical Review Letters*, 108(13), 130503.
4. van Assche, G. (2006). *Quantum cryptography and secrete-key distillation*. Cambridge University Press.
5. Liao, S.-K., et al. (September 7, 2017). Satellite-to-ground quantum key distribution. *Nature*, 549, 43–44.
6. Djordjevic, I. B. (2019). *Physical-layer security and quantum key distribution*. Springer Nature Switzerland AG.
7. Duan, L.-M., Lukin, M. D., Cirac, J. I., & Zoller, P. (2001). Long-distance quantum communication with atomic ensembles and linear optics. *Nature*, 414(6862), 413–418.
8. Qiu, J. (2014). Quantum communications leap out of the lab. *Nature*, 508(7497), 441–442.
9. Lucamarini, M., Yuan, Z. L., Dynes, J. F., & Shields, A. J. (2018). Overcoming the rate–distance limit of quantum key distribution without quantum repeaters. *Nature*, 557, 400–403.
10. Ma, X., Zeng, P., & Zhou, H. (2018). Phase-matching quantum key distribution. *Physical Review X*, 8, 031043.
11. Fang, X.-T., Zeng, P., Liu, H., et al. (2020). Implementation of quantum key distribution surpassing the linear rate-transmittance bound. *Nature Photonics*, 14, 422–425.
12. Bacco, D., Da Lio, B., Cozzolino, D., et al. (2019). Boosting the secret key rate in a shared quantum and classical fibre communication system. *Communications Physics*, 2, 140.
13. McEliece, R. J. (1978). A public-key cryptosystem based on algebraic coding theory. In *DNS progress report* (pp. 114–116). Jet Propulsion Laboratory.
14. Bernstein, D. J., Buchmann, J., & Dahmen, E. (2009). *Post-quantum cryptography*. Springer.
15. Branco, P., Mateus, P., Salema, C., & Souto, A. (2020). Using low-density parity-check codes to improve the McEliece cryptosystem. *Information Sciences*, 510, 243–255.
16. NIST, Post-quantum cryptography PQC. Retrieved from <https://csrc.nist.gov/projects/post-quantum-cryptography>
17. Micciancio, D., & Regev, O. (2009). Lattice-based cryptography. In D. J. Bernstein, J. Buchmann, & E. Dahmen (Eds.), *Post-quantum cryptography*. Springer.
18. Regev, O. (08 September 2009). On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6), 1–40.
19. Lyubashevsky, V., Peikert, C., & Regev, O. (2010). On ideal lattices and learning with errors over rings. In H. Gilbert (Ed.), *Advances in cryptology—EUROCRYPT 2010* (Vol. 6110, pp. 1–23, Lecture Notes in Computer Science,). Springer-Verlag.

20. Harrow, A. W., Hassidim, A., & Lloyd, S. (2009). Quantum algorithm for solving linear systems of equations. *Physical Review Letters*, 103, 150502.
21. Djordjevic, I. B. (2020). Joint QKD-post-quantum cryptosystems. *IEEE Access*, 8, 154708–154712.
22. Buchanan, W. J. *Public key encryption with learning with errors (LWE)*. Retrieved from <https://asecuritysite.com/encryption/lwe2>
23. Goldreich, O., & Goldwasser, S. (2000). On the limits of nonapproximability of lattice problems. *Journal of Computer and System Sciences*, 60(3), 540–563.
24. Ajtai, M. (2005). Representing hard lattices with $O(n \log n)$ bits. *Proceeding of the 37th Annual ACM Symp. on Theory of Computing (STOC)* (pp. 94–103). Baltimore, MD, USA, May 22–24, 2005.
25. Lyubashevsky, V., & Micciancio, D. (2008). Asymptotically efficient lattice-based digital signatures. *Fifth Theory of Cryptography Conference (TCC)*, volume 4948 of Lecture Notes in Computer Science. Springer.
26. Micciancio, D. (01 January 2009). Cryptographic functions from worst-case complexity assumptions. In P. Q. Nguyen & B. Vallée (Eds.), *The LLL algorithm: Survey and applications* (pp. 427–452). Springer.
27. Lyubashevsky, V., Micciancio, D., Peikert, C., & Rosen, A. (2008). SWIFFT: A modest proposal for FFT hashing. *Proceeding of the 15th International Workshop Fast Software Encryption (FSE 2008)* (pp. 54–72). Lausanne, Switzerland, February 10–13, 2008.
28. Peikert, C. (2016). *A decade of lattice cryptography*, now, <https://doi.org/10.1561/04000000074>.
29. Baldi, M., Chiaraluce, F., & Bianchi, M. (2013). Security and complexity of the McEliece cryptosystem based on quasi-cyclic low-density parity-check codes. *IET Information Security*, 7(3), 212–220.
30. Sun, X., Zou, D., Qu, Z., & Djordjevic, I. (2018). Run-time reconfigurable adaptive LDPC coding for optical channels. *Optics Express*, 26(22), 29319–29329.
31. Niederreiter, H. (1986). Knapsack-type cryptosystems and algebraic coding theory. *Problems of Control and Information Theory (Problemy Upravljenija i Teorii Informacii)*, 15, 159–166.
32. Djordjevic, I. B., et al. (2008). Large girth low-density parity-check codes for long-haul high-speed optical communications. *Proceeding of the OFC/NFOEC 2008*, San Diego, CA, Paper no. JWA53.
33. Djordjevic, I. B. (2019). *Physical-layer security and quantum key distribution*. Springer International Publishing.
34. Sun, X., & Djordjevic, I. B. (2019). FPGA implementation of rate-adaptive spatially-coupled LDPC codes suitable for optical communications. *Optics Express*, 27(3), 3422–3428.
35. Bernstein, D. J. (2010). Grover vs. McEliece. In N. Sendrier (Ed.), *Post-quantum cryptography (PQCrypto 2010)* (Lecture Notes in Computer Science) (Vol. 6061). Springer.
36. Tamura, Y., et al. (2018). The First 0.14-dB/km loss optical fiber and its impact on submarine transmission. *Journal of Lightwave Technology*, 36, 44–49.
37. Qu, Z., & Djordjevic, I. B. (2017). Four-dimensionally multiplexed eight-state continuous-variable quantum key distribution over turbulent channels. *IEEE Photonics Journal*, 9(6), 7600408.
38. Lamport, L. (18 October 1979). *Constructing digital signatures from a one way function*. SRI International, CSL-98, Op. 52.
39. Merkle, R. C. (1979). *Secrecy, authentication and public key systems*. Ph.D. Dissertation. Stanford University.
40. Merkle, R. C. (1989). A certified digital signature. *Proceeding of the Conference on the Theory and Application of Cryptology (CRYPTO '89)* (pp. 218–238). Springer.
41. Paar, C., Pelzl, J., & Güneysu, T. (2024). *Understanding cryptography: From established symmetric and asymmetric ciphers to post-quantum algorithms* (2nd ed.). Springer Verlag GmbH.
42. Aumasson, J.-P. (2018). *Serious cryptography: A practical introduction to modern encryption*. No Starch Press.

43. Sakumoto, K., Shirai, T., & Hiwatari, H. (2011). Public-Key Identification Schemes Based on Multivariate Quadratic Polynomials. *Proceeding of the Advances in Cryptology – CRYPTO, 2011*, 706–723.
44. Yasuda, T., Wang, Y., & Takagi, T. (2020). Multivariate encryption schemes based on polynomial equations over real numbers. *Proceeding of the International Conference on Post-Quantum Cryptography PQCrypto, 2020*, 402–421.
45. Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability; A guide to the theory of NP-completeness*. W. H. Freeman & Co..
46. Ding, J., Gower, J. E., & Schmidt, D. S. (2006). *Multivariate public key cryptosystems*. Springer.
47. Dubois, V., Fouque, P.-A., Shamir, A., & Stern, J. (2007). Practical cryptanalysis of SFLASH. In A. Menezes (Ed.), *CRYPTO 2007* (Vol. 4622, pp. 1–12). LNCS, Springer.
48. Kipnis, A., & Shamir, A. (1998). Cryptanalysis of the oil & vinegar signature scheme. In H. Krawczyk (Ed.), *CRYPTO 1998* (Vol. 1462, pp. 257–266). LNCS, Springer.